UNIVERSIDAD NACIONAL AUTÓNOMA DE NICARAGUA, LEÓN

ÁREA DE CONOCIMIENTO DE CIENCIAS Y TECNOLOGÍA
INGENIERÍA EN SISTEMAS DE INFORMACIÓN

BODETIME

Sistema para el Control de Horarios de Entrada y Salida de Trabajadores

Proyecto Integrador II

Integrantes: Ángel Muñoz y Leonardo Pichardo
Carrera: Ingeniería en Sistemas de Información
Año: Segundo | Grupo: 2
Tutor: [Completar con el nombre oficial del tutor]
León, Nicaragua • 2026

DOCUMENTO REVISADO Y ALINEADO CON LA RÚBRICA DE PRE-DEFENSA

Nota de alcance y veracidad

Este documento reorganiza y fortalece la propuesta BODETIME a partir del proyecto entregado y de los requisitos de la rúbrica de pre-defensa. La documentación original declara Python, PySide6 y SQLite; la rúbrica, en cambio, exige demostrar MySQL mediante SQLAlchemy, CRUD completo, interfaz funcional y generación de PDF integrada. Por esa razón, esta versión define la arquitectura objetivo compatible con la rúbrica y distingue lo que está documentado de lo que debe comprobarse en el software.

Aspecto	Lo que aparece en el documento base	Requisito de evaluación / ajuste
Persistencia	SQLite local.	Migrar y demostrar MySQL con SQLAlchemy ORM.
Interfaz	PySide6 descrito a nivel general.	Demostrar ventanas reales, navegación y QMessageBox.
CRUD	No se documentan las cuatro operaciones completas.	Implementar y probar crear, consultar, actualizar y eliminar sobre MySQL.
PDF	Reportes aparecen como función futura.	Generar PDF desde un botón de la GUI usando ReportLab.
Resultados	Se redactan principalmente como esperados.	Sustituir por resultados observados y evidencias luego de ejecutar las pruebas.

Importante: no se afirma que las funciones anteriores ya estén implementadas ni probadas. Antes de la defensa deben ejecutarse en el equipo y actualizarse las evidencias. No se proporcionó el código fuente, por lo que no es posible certificar la ejecución real ni modificar internamente la aplicación.

Resumen ejecutivo

BODETIME es una aplicación de escritorio orientada al registro y consulta de la asistencia del personal de una bodega. Su propósito es centralizar los datos de trabajadores y marcajes de entrada y salida, facilitar la identificación de tardanzas según un horario configurable y producir reportes consultables. La solución propuesta utiliza Python para la lógica, PySide6 para la interfaz, MySQL como gestor de base de datos, SQLAlchemy como ORM y ReportLab para generar reportes PDF desde la aplicación. El desarrollo se organiza mediante Scrum, con entregas incrementales, pruebas funcionales y revisión de cada incremento.

1. Introducción

En una bodega, el control de asistencia permite conocer quién se presentó a trabajar y a qué hora realizó su marcaje. Cuando esta información se lleva únicamente en registros manuales, pueden presentarse errores de transcripción, pérdida de hojas, consultas lentas y dificultades para revisar el historial. BODETIME plantea digitalizar ese proceso mediante una aplicación de escritorio que almacene los datos de manera estructurada y permita consultarlos con rapidez.

La aplicación se diseña para que el usuario registre trabajadores, marque entradas y salidas, consulte los registros y genere reportes. La hora del marcaje debe obtenerse desde el reloj del sistema en el momento de la operación; el horario laboral y la tolerancia de tardanza deben ser configurables por el responsable, no valores supuestos en el informe.

2. Planteamiento del problema

Problema central: falta de un mecanismo digital centralizado y verificable para registrar y consultar los horarios de entrada y salida de los trabajadores de una bodega.

2.1 Pregunta del problema

¿Cómo puede una aplicación de escritorio facilitar el registro, consulta y seguimiento de la asistencia de los trabajadores, reduciendo errores asociados al control manual?

2.2 Causas y efectos (Ishikawa)

Categoría	Causas posibles	Efecto en el proceso
Método	Registro manual; ausencia de un procedimiento uniforme.	Diferencias entre registros y dificultad para verificar horarios.
Personas	Dependencia de quien anota; errores al escribir nombres u horas.	Datos incompletos o inconsistentes.
Tecnología	No existe una herramienta centralizada o respaldos sistemáticos.	Pérdida de información y consultas lentas.
Información	Formatos distintos y falta de historial organizado.	Dificultad para elaborar reportes y detectar tardanzas.
Control	Horario y criterios de tardanza no documentados.	Evaluaciones no uniformes de puntualidad.

El diagrama de Ishikawa organiza hipótesis de causas; no demuestra por sí solo que todas ocurran en la bodega. Deben validarse mediante observación o entrevista con el usuario del proceso.

3. Objetivos

3.1 Objetivo general

Desarrollar una aplicación de escritorio que permita gestionar trabajadores y controlar sus registros de asistencia mediante operaciones CRUD, almacenamiento en MySQL y generación de reportes PDF.

3.2 Objetivos específicos

- Registrar, consultar, actualizar y eliminar la información de los trabajadores.
- Registrar las horas de entrada y salida asociadas a cada trabajador.
- Consultar los marcajes por trabajador y por rango de fechas.
- Identificar tardanzas comparando el marcaje con un horario configurable.
- Generar reportes PDF desde la interfaz gráfica con datos recuperados de MySQL.
- Validar los datos ingresados y comunicar errores o confirmaciones mediante cuadros de diálogo.
- Aplicar pruebas funcionales para comprobar la persistencia y consistencia de la información.

4. Justificación

La propuesta responde a la necesidad de organizar los datos de asistencia en un único sistema. La digitalización facilita la búsqueda de registros, reduce la dependencia de anotaciones dispersas y permite generar reportes con criterios uniformes. Desde el ámbito académico, integra programación orientada a objetos, interfaces gráficas, bases de datos relacionales, persistencia mediante ORM, generación de documentos y gestión ágil del proyecto.

5. Alcance y delimitaciones

5.1 Alcance funcional propuesto para la pre-defensa

- Mantenimiento de trabajadores: alta, consulta, modificación y baja lógica o eliminación controlada.
- Registro de asistencia: entrada y salida con fecha y hora generadas automáticamente.
- Consulta de asistencia por fecha, trabajador y estado.
- Cálculo de tardanza con base en hora de inicio y tolerancia configurables.
- Reporte PDF de asistencia con filtros y fecha de generación.
- Validaciones de campos obligatorios, formatos, duplicados y secuencia entrada/salida.

5.2 Fuera del alcance de esta versión

- Relojes biométricos, tarjetas RFID o integración con dispositivos externos.
- Cálculo de nómina, deducciones salariales o gestión de vacaciones.
- Acceso remoto multi-sucursal y sincronización en la nube.
- Aplicación móvil y notificaciones automáticas.

Estas exclusiones evitan prometer funciones que no se desprenden de la rúbrica ni del alcance inicial.

6. Marco teórico

6.1 Sistema de información

Conjunto organizado de recursos que captura, procesa, almacena y presenta información para apoyar actividades y decisiones. BODETIME transforma marcajes y datos del personal en registros consultables.

6.2 Control de asistencia

Proceso de registrar la presencia de las personas y los momentos de entrada y salida. La calidad del control depende de reglas claras, datos íntegros y trazabilidad.

6.3 Base de datos relacional

Modelo que organiza información en tablas relacionadas mediante claves primarias y foráneas. La relación entre trabajadores y asistencias permite conservar el historial de cada persona.

6.4 ORM y SQLAlchemy

Un mapeador objeto-relacional vincula clases de Python con tablas de la base de datos. SQLAlchemy permite definir modelos, administrar sesiones y ejecutar operaciones persistentes sin construir manualmente cada sentencia SQL.

6.5 Interfaz gráfica con PySide6

PySide6 ofrece bindings de Qt para Python. Sus widgets, señales y slots permiten construir formularios, tablas, botones y cuadros de diálogo para la interacción con el usuario.

6.6 Generación de PDF con ReportLab

ReportLab permite construir documentos PDF en Python. Su módulo Platypus organiza contenido mediante elementos como párrafos y tablas; TableStyle controla encabezados, bordes, alineación y espaciado.

6.7 Validación y transacciones

La validación comprueba datos antes de guardarlos. Las transacciones agrupan operaciones para confirmar todos los cambios o revertirlos cuando ocurre un error, evitando estados parciales.

6.8 Scrum

Scrum es un marco de trabajo para desarrollar productos en incrementos. El equipo mantiene un Product Backlog, selecciona trabajo para cada Sprint, revisa el incremento y analiza oportunidades de mejora.

7. Metodología de desarrollo: Scrum

Se conserva la organización de roles propuesta en el documento base. Como el equipo está formado por dos integrantes, ambos participan en el desarrollo y colaboran en las pruebas; los roles describen responsabilidades, no puestos exclusivos.

Integrante	Responsabilidad Scrum	Actividades principales
Ángel Muñoz	Product Owner y Developer	Representar necesidades, priorizar el backlog, revisar criterios de aceptación y desarrollar funcionalidades acordadas.
Leonardo Pichardo	Scrum Master y Developer	Facilitar la organización del trabajo, dar seguimiento a obstáculos, apoyar la calidad e implementar funcionalidades.

7.1 Eventos

Evento	Propósito	Evidencia sugerida
Sprint Planning	Definir objetivo y seleccionar tareas del Sprint.	Objetivo, tareas, responsables y estimación.
Daily Scrum	Inspeccionar avance y obstáculos.	Actualización breve del tablero.
Sprint Review	Mostrar el incremento funcional y recibir comentarios.	Demostración de la aplicación.
Sprint Retrospective	Identificar qué mejorar en el siguiente Sprint.	Acuerdos de mejora.

7.2 Product Backlog priorizado

ID	Historia / funcionalidad	Prioridad	Criterio de aceptación resumido
PB-01	Gestionar trabajadores (CRUD).	Alta	Se crea, consulta, actualiza y elimina/desactiva un trabajador en MySQL.
PB-02	Registrar entrada.	Alta	Guarda trabajador, fecha y hora reales; evita marcajes inválidos.
PB-03	Registrar salida.	Alta	Completa el marcaje abierto del trabajador; no admite salida sin entrada.
PB-04	Consultar asistencia.	Alta	Filtra y muestra datos persistidos por fecha y trabajador.
PB-05	Calcular tardanza.	Alta	Compara hora de entrada

			con horario y tolerancia configurados.
PB-06	Generar PDF desde GUI.	Alta	El botón genera un PDF con los resultados del filtro actual.
PB-07	Validar entradas y mostrar mensajes.	Alta	Los errores se explican con QMessageBox sin cerrar inesperadamente la app.
PB-08	Administrar usuarios y permisos.	Media	Acceso protegido por roles, si se implementa dentro del alcance acordado.

7.3 Sprint backlog sugerido

Sprint	Objetivo	Tareas verificables
Sprint 1	Base técnica y trabajadores	Crear esquema MySQL; configurar SQLAlchemy; implementar modelo Trabajador; CRUD; validaciones.
Sprint 2	Marcajes y consultas	Modelo Asistencia; entrada/salida; reglas de duplicidad; filtros; pruebas de integridad.
Sprint 3	Reportes y cierre	Reporte PDF con ReportLab; integración al botón; pruebas completas; corrección de interfaz; preparación de defensa.

Las fechas del Sprint 1 que aparecen en el documento original (27/08/2026 al 01/09/2026) se conservan como antecedente del plan inicial, pero no se presentan como evidencia de que cada tarea haya sido completada. Actualizar el tablero y las fechas con el trabajo realmente realizado.

8. Requisitos del sistema

8.1 Requisitos funcionales

ID	Requisito
RF-01	El sistema debe permitir registrar trabajadores con nombre y datos básicos definidos.
RF-02	El sistema debe mostrar la lista de trabajadores almacenados en MySQL.
RF-03	El sistema debe permitir actualizar la información de un trabajador.
RF-04	El sistema debe permitir eliminar o desactivar un trabajador con confirmación.
RF-05	El sistema debe registrar la hora de entrada usando fecha y hora del sistema.
RF-06	El sistema debe registrar la hora de salida vinculada a una entrada abierta.
RF-07	El sistema debe consultar asistencia por trabajador y rango de fechas.
RF-08	El sistema debe identificar tardanzas según horario y tolerancia configurables.
RF-09	El sistema debe generar un PDF desde un botón de la

	interfaz gráfica.
RF-10	El sistema debe mostrar mensajes claros ante datos inválidos y operaciones exitosas.

8.2 Requisitos no funcionales

ID	Requisito
RNF-01	La interfaz debe ser consistente, legible y fácil de navegar.
RNF-02	La persistencia debe realizarse en MySQL mediante SQLAlchemy ORM.
RNF-03	Las operaciones de escritura deben utilizar transacciones y manejo de errores.
RNF-04	Las credenciales de conexión no deben quedar escritas directamente en el código fuente.
RNF-05	Los campos y relaciones deben proteger la integridad de los datos.
RNF-06	La aplicación debe informar si no puede conectarse a la base de datos.
RNF-07	El código debe estar separado por responsabilidades para facilitar mantenimiento.
RNF-08	El reporte debe mostrar fecha de generación, filtros aplicados y encabezados legibles.

9. Diseño técnico de la solución

9.1 Arquitectura por capas

Capa	Tecnología	Responsabilidad
Presentación	PySide6	Ventanas, formularios, tablas, navegación y QMessageBox.
Aplicación / servicios	Python	Reglas de asistencia, validaciones, coordinación de operaciones y filtros.
Persistencia	SQLAlchemy ORM	Modelos, sesiones, transacciones y consultas.
Datos	MySQL	Almacenamiento persistente, claves, restricciones e índices.
Reportes	ReportLab	Construcción del PDF con datos consultados desde la capa de servicios.

Flujo: Usuario → PySide6 → Servicio de aplicación → SQLAlchemy Session → MySQL. Para reportes: PySide6 → Servicio de reportes → consulta de datos → ReportLab → archivo PDF.

9.2 Modelo relacional propuesto

Tabla	Campos principales	Reglas
trabajadores	id_trabajador (PK), codigo (UNIQUE), nombres, apellidos, cargo, telefono, activo, creado_en	codigo único; nombres y apellidos obligatorios.
asistencias	id_asistencia (PK), id_trabajador (FK), fecha, hora_entrada, hora_salida,	FK a trabajadores; fecha obligatoria; salida posterior a entrada.

	estado, observacion	
configuracion_horario	id_configuracion (PK), hora_inicio, tolerancia_minutos, vigente_desde	Define la regla que se usa para clasificar tardanzas.

Relación: un trabajador puede tener muchas asistencias (1:N). La asistencia pertenece a un único trabajador. Para evitar más de un registro diario por persona, puede aplicarse una restricción UNIQUE(id_trabajador, fecha), si la operación real no contempla turnos múltiples. Si hay turnos múltiples, debe añadirse una entidad Turno y ajustar la clave única.

9.3 Reglas de negocio

- No se registra asistencia para un trabajador inactivo.
- La entrada se genera con la fecha y hora del sistema; el usuario no debe poder alterar el marcaje ordinario.
- No se permite registrar una segunda entrada diaria abierta para el mismo trabajador.
- La salida solo se registra si existe una entrada sin salida.
- La hora de salida debe ser posterior a la hora de entrada.
- La tardanza se calcula comparando la hora de entrada con la hora de inicio más la tolerancia definida.
- Toda eliminación debe confirmar la acción; para conservar historial, se prefiere baja lógica del trabajador.

9.4 Tecnologías

Tecnología	Uso en BODETIME
Python	Lenguaje principal para lógica y reglas.
PySide6	Interfaz gráfica de escritorio basada en Qt.
MySQL	Base de datos relacional exigida para la demostración.
SQLAlchemy ORM	Mapeo de clases a tablas y operaciones CRUD.
PyMySQL	Driver DBAPI para conectar SQLAlchemy con MySQL.
ReportLab	Generación del PDF desde la aplicación.
Visual Studio Code	Edición y depuración del código.
Git	Control de versiones y trazabilidad de cambios.
Scrum	Organización incremental del trabajo.

10. Diseño de interfaz y navegación

La interfaz debe mantener una navegación simple y predecible. Se recomienda una ventana principal con menú lateral o barra superior y módulos claramente identificados: Inicio, Trabajadores, Asistencia, Reportes y Configuración. Cada tabla debe mostrar encabezados claros, permitir actualizar la vista tras una operación y ofrecer mensajes de confirmación.

Vista	Elementos mínimos	Validaciones / mensajes
Inicio	Resumen de trabajadores activos, marcajes del día y tardanzas.	Aviso visible si la base de datos no está disponible.
Trabajadores	Formulario y tabla con botones Nuevo, Guardar, Editar, Eliminar y Limpiar.	Campos requeridos, código duplicado y confirmación antes de eliminar.
Asistencia	Selector de trabajador, botones Entrada/Salida y tabla de registros.	Evitar duplicados, salida sin entrada y salida anterior a entrada.
Reportes	Filtros por fecha/trabajador y botón	Aviso si no hay datos; confirmar ruta

	Generar PDF.	o mostrar dónde se guardó el archivo.
Configuración	Hora de inicio y tolerancia.	Validar formato de hora y valores no negativos.

11. Plan de pruebas y criterios de aceptación

Las pruebas siguientes son casos que deben ejecutarse en la aplicación real. El resultado esperado no equivale a un resultado ya obtenido. Durante la pre-defensa, mostrar al jurado al menos una prueba positiva y una negativa por módulo.

ID	Prueba / acción	Resultado esperado	Evidencia
T-01	Crear trabajador con datos válidos.	Se guarda en MySQL y aparece en la tabla.	Captura GUI y fila consultada.
T-02	Consultar trabajador existente.	Se muestran sus datos actuales desde MySQL.	Captura de tabla.
T-03	Editar un dato y guardar.	El cambio persiste al recargar la vista.	Antes/después.
T-04	Eliminar/desactivar trabajador.	Pide confirmación y aplica la regla definida.	Mensaje y consulta posterior.
T-05	Guardar trabajador con campos vacíos.	Bloquea operación y muestra QMessageBox.	Captura del mensaje.
T-06	Registrar entrada.	Guarda fecha y hora y actualiza la tabla.	Marcaje en GUI y BD.
T-07	Registrar entrada duplicada.	Rechaza el marcaje según regla diaria.	Mensaje de validación.
T-08	Registrar salida sin entrada.	Rechaza la operación.	Mensaje de validación.
T-09	Registrar salida válida.	Completa el registro y persiste en MySQL.	Fila con entrada/salida.
T-10	Generar reporte desde botón.	Crea PDF con datos reales y encabezados.	PDF generado y abierto.
T-11	Filtrar rango sin resultados.	Muestra aviso y no genera un reporte engañoso.	Mensaje.
T-12	Desconectar MySQL y abrir módulo.	La aplicación informa el problema sin cerrarse.	Mensaje de conexión.

12. Resultados y discusión

El material recibido describe el objetivo, la planificación y el diseño conceptual de BODETIME, pero no contiene código fuente, capturas de la aplicación en ejecución, registros de pruebas ni un PDF generado desde la interfaz. Por tanto, no es posible afirmar resultados de ejecución ni declarar aprobado el CRUD. Para cerrar esta sección, el equipo debe incorporar resultados observados con evidencias y sustituir los campos de la tabla por datos reales.

Indicador	Resultado observado	Evidencia a adjuntar
CRUD de trabajadores	[Completar después de ejecutar]	Capturas crear, leer, actualizar y eliminar.
Persistencia MySQL	[Completar después de ejecutar]	Consulta de registros en MySQL Workbench.

Marcajes entrada/salida	[Completar después de ejecutar]	Captura de marcaje y fila persistida.
Validaciones visuales	[Completar después de ejecutar]	Capturas QMessageBox.
PDF en vivo	[Completar después de ejecutar]	PDF generado desde el botón y captura de la GUI.

Discusión: la arquitectura propuesta separa interfaz, reglas, persistencia y reportes, lo que facilita las pruebas y el mantenimiento. El cambio de SQLite a MySQL no es solo editorial: requiere configurar el servidor, crear el esquema, definir modelos ORM y comprobar las cuatro operaciones CRUD contra datos reales. Asimismo, el PDF debe construirse con datos consultados en el momento, no con contenido estático.

13. Conclusiones

- BODETIME aborda una necesidad concreta: organizar y consultar el registro de asistencia de trabajadores de una bodega.
- La solución debe mantener coherencia entre los requisitos, el modelo relacional y las funciones demostradas.
- Para alinearse con la rúbrica, la implementación debe utilizar MySQL con SQLAlchemy, presentar CRUD completo y generar el PDF desde la interfaz mediante ReportLab.
- La metodología Scrum ofrece una estructura para priorizar tareas, inspeccionar incrementos y corregir problemas antes de la defensa.
- La validez de los resultados depende de ejecutar las pruebas y adjuntar evidencias reales; la documentación por sí sola no demuestra que el software funcione.

14. Recomendaciones

- Priorizar la migración comprobable a MySQL y SQLAlchemy antes de agregar funciones secundarias.
- Ensayar la demostración completa en el mismo equipo y entorno que se utilizará en la pre-defensa.
- Preparar datos de prueba ficticios y consistentes, sin información personal real de trabajadores.
- Respaldar la base de datos y mantener las credenciales fuera del repositorio.
- Adjuntar capturas de las operaciones CRUD, validaciones y PDF generado desde la GUI.
- No presentar como implementadas las funciones que aún estén pendientes.
- Completar el nombre del tutor y verificar los nombres oficiales de los integrantes antes de imprimir.

15. Guion técnico para la demostración

Orden	Acción	Qué explicar al jurado
1	Abrir BODETIME y mostrar la ventana principal.	La GUI está desarrollada con PySide6 y se comunica con la capa de servicios.
2	Crear un trabajador de prueba.	Se valida el formulario y SQLAlchemy persiste el objeto en MySQL.
3	Consultar y editar el trabajador.	La tabla refleja los datos recuperados de la base de datos.
4	Eliminar o desactivar el registro de prueba.	La operación se confirma y se respeta la integridad referencial.
5	Marcar entrada y luego salida.	El sistema guarda fecha y hora y aplica las reglas de negocio.

6	Mostrar una validación negativa.	QMessageBox informa el error sin provocar un cierre inesperado.
7	Filtrar y generar el PDF.	ReportLab construye el documento con los resultados reales de la consulta.

16. Preguntas probables del jurado

¿Por qué MySQL y no SQLite?

Porque la rúbrica solicita explícitamente demostrar persistencia en MySQL. SQLite fue la tecnología descrita en el documento inicial, pero debe sustituirse para esta evaluación.

¿Qué es SQLAlchemy ORM?

Es la capa que relaciona clases de Python con tablas de la base de datos y permite realizar operaciones persistentes mediante sesiones.

¿Qué significa CRUD?

Create, Read, Update y Delete: crear, consultar, actualizar y eliminar.

¿Cómo se calcula una tardanza?

Se compara la hora de entrada registrada con la hora de inicio del turno más la tolerancia configurada.

¿Cómo se evita una salida sin entrada?

La regla de negocio verifica que exista una asistencia abierta antes de guardar la salida.

¿Cómo se genera el PDF?

La GUI solicita el reporte a un servicio; este consulta MySQL y ReportLab construye el archivo con encabezados, filas y formato.

¿Qué función tiene Scrum?

Organiza el trabajo en incrementos cortos, permite priorizar funcionalidades y revisar avances con el equipo.

17. Referencias técnicas

- SQLAlchemy. MySQL and MariaDB — SQLAlchemy 2.0 Documentation. https://docs.sqlalchemy.org/en/20/dialects/mysql.html
- SQLAlchemy. ORM Quick Start. https://docs.sqlalchemy.org/en/20/orm/quickstart.html
- Qt for Python. PySide6 Documentation. https://doc.qt.io/qtforpython-6/
- Qt for Python. QMessageBox. https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QMessageBox.html
- ReportLab. Chapter 7: Tables and TableStyles. https://docs.reportlab.com/reportlab/userguide/ch7_tables/
- ReportLab. ReportLab User Guide. https://docs.reportlab.com/reportlab/userguide/
- Scrum Guides. The Scrum Guide. https://scrumguides.org/scrum-guide.html

Fuentes institucionales consultadas para fundamentar la arquitectura. Los ejemplos y decisiones de diseño de este documento son una propuesta adaptada al proyecto; no sustituyen la implementación ni las pruebas del equipo.